Join our community, grow your knowledge and learn from others!
Sign In
Don't have an Epic Games account? Sign up

Actor Role and Remote Role

Determine how much control this game instance has over an actor.

In networked gameplay, an actor's **role** and **remote role** determine which machine has the authority to change the actor's state and perform remote procedure calls. These properties help you answer the following questions about an actor in your multiplayer game:

- Is this actor replicated or not?
- Who has authority over this actor?
- What is the replication role for this actor?

> Actor role and remote role are not the same as ownership! For more information, see Actors and their Owning Connections.

View an Actor's Role and Remote Role

You can see the current role and remote role of an actor in the Unreal Editor's Details panel:

Sample Role and Remote Role

An actor's role and remote role can also be obtained in C++ or Blueprints.

Get Actor's Role

To determine how much control the local machine has over this actor, call AActor::GetLocalRole:

```
C++

AMyActor* MyActor;
...
// Once you have a valid Actor pointer...
const ENetRole LocalRole = MyActor->GetLocalRole();
// LocalRole contains the Actor's Role
...
```

Copy full snippet (6 lines long)

Get Actor's Remote Role

To determine how much control a remote machine has over this actor, call AActor::GetRemoteRole:

```
C++

AMyActor* MyActor;
...
// Once you have a valid Actor pointer...
const ENetRole RemoteRole = MyActor->GetRemoteRole();
// RemoteRole contains the Actor's RemoteRole
...
```

Copy full snippet (6 lines long)

Actor Role States

Actor role and remote role are represented by a state of the `ENetRole` enumeration class (defined in `EngineTypes.h`). The following table lists and describes these states:

Net Role State	Description
`ROLE_None`	No role. This actor is not replicated.
`ROLE_SimulatedProxy`	Simulated proxy of this actor. This actor simulates the true state but has no authority to change the state or call remote functions.
`ROLE_AutonomousProxy`	Autonomous proxy of this actor. This actor simulates the true state and has the authority to change the state and call remote functions.
`ROLE_Authority`	Authoritative control of this actor. This actor holds the true state for networking and has the authority to change the state and call functions. This actor is also in charge of keeping track of property changes and replicating them to connected clients.

Matrix of Roles for Client-Server

Unreal Engine uses the client-server model for networking. As a result, the server always has authority over replicated actors. This means that only the server should ever see `ROLE_Authority` for replicated actors. Non-replicated actors can have a local role of `ROLE_Authority` on clients and a remote role of `ROLE_None`.

The following table provides a matrix of local role and remote role, whether a server or client is witnessing this role combination, and a description of what this combination means:

Local Role	Remote Role	Server or Client	Description
`ROLE_Authority`	`ROLE_AutonomousProxy`	Server	This actor pawn is controlled by one of the connected clients.
`ROLE_AutonomousProxy`	`ROLE_Authority`	Client	This actor pawn is controlled by this connected client.
`ROLE_SimulatedProxy`	`ROLE_Authority`	Client	This actor pawn is controlled by one of the other connected clients. Replicated actors that are not controlled by any client can also have this role combination.
`ROLE_Authority`	`ROLE_None`	Client	This is a non-replicated actor.

Actor Replication Simulation

Servers do not replicate all actors every update because that would take too much bandwidth and CPU resources. Instead, the server replicates actors at the frequency specified on the AActor::NetUpdateFrequency property. This means that some time passes on the client between actor updates. This could result in actors with a character movement component looking choppy and sporadic. To compensate for this, the client simulates character movement between updates.

For more information about replication simulation and character movement, see Networked Movement in the Character Movement Component documentation.

Role and Remote Role Reference

Functions

Name	Description
CopyRemoteRoleFrom	Copy remote role from another actor and add this actor to the list of network actors if necessary.
ExchangeNetRoles	Swap role and remote role if on client.
GetLocalRole	Return how much control the local machine has over this actor.
GetRemoteRole	Return how much control the remote machine has over this actor.
GetRoleProperty Name	Get the property name for role.
GetTearOff	If true, this actor is no longer replicated to new clients and is "torn off" (becomes `ENetRole::ROLE_Authority`) on clients to which it was being replicated.
PostNetReceiveRole	Called immediately after a new role is received from the remote machine.
SetRemoteRoleFor BackwardsCompat	Used in the constructor of classes that must set the remote role for backwards compatibility purposes.
SetRole	Set the value of role without causing other side effects to this instance.
SwapRoles	Swap the role and remote role for this actor. Use caution and only call this function if you understand the side effects.

Properties

Name	Description
bExchangedRoles	Whether role and remote role have been exchanged on the client.

programming networking role remote role simulated autonomous authority

Ask questions and help your peers Developer Forums
Write your own tutorials or read those from others Learning Library

ON THIS PAGE
View an Actor's Role and Remote Role
Get Actor's Role
Get Actor's Remote Role
Actor Role States
Matrix of Roles for Client-Server
Actor Replication Simulation
Role and Remote Role Reference
Functions
Properties